
MODULE *FormalizedCorollary*

EXTENDS *Integers, FiniteSets, Sequences, Utils*

Segments

$Segment(a, b) \triangleq \langle a, b \rangle$

$Seg \triangleq \{s \in Int \times Int : s[1] \leq s[2]\}$

$EmptySeg \triangleq \langle \rangle$ the empty segment is not part of *Seg*

$b(s) \triangleq s[1]$

$e(s) \triangleq s[2]$

$SegShiftLeft(s) \triangleq Segment(b(s) - 1, e(s) - 1)$

$SegTruncateLeft(s) \triangleq$
 IF $b(s) = e(s)$
 THEN $EmptySeg$
 ELSE $Segment(b(s) + 1, e(s))$

$Precedes(s1, s2) \triangleq$
 returns $s1 \ s2$
 $\wedge b(s1) < b(s2)$
 $\wedge e(s1) < e(s2)$

$SegSubsetEq(s1, s2) \triangleq$
 checks if $s1$ is a subset of $s2$
 $\wedge b(s2) \leq b(s1)$
 $\wedge e(s1) \leq e(s2)$

Multisegments

$Multisegments \triangleq Seq(Seg)$

$SubMultisegment(N, M) \triangleq SubBag(N, M)$

$$\text{SameMultisegment}(M, N) \triangleq \\ \text{SameBag}(M, N)$$

$$\text{IndexSet}(M) \triangleq \\ 1 \dots \text{Len}(M)$$

$$\text{Delta}(M) \triangleq \\ \text{Delta}(M) \text{ is the function } i \mapsto \text{Delta}_i \\ [i \in \text{IndexSet}(M) \mapsto M[i]]$$

$$\text{IsLadder}(M) \triangleq \\ \wedge M \neq \langle \rangle \quad \text{a ladder is nonempty} \\ \wedge \exists R \in \text{Permutations}(\text{IndexSet}(M)) : \\ \forall i \in 1 \dots (\text{Len}(M) - 1) : \\ \text{Precedes}(M[R[i + 1]], M[R[i]]) \quad \text{Delta}_{\{R[i + 1]\}} \text{Delta}_{\{R[i]\}}$$

$$\text{Depth}$$

$$\text{Depth}(M) \triangleq \\ \text{LET} \\ \text{IsDepthChain}(i, j, \text{Chain}) \triangleq \\ \wedge \text{Chain}[1] = i \quad \text{the chain starts at index } i \\ \wedge \forall r \in 1 \dots j : \\ \text{Precedes}(\\ \text{Delta}(M)[\text{Chain}[r]], \\ \text{Delta}(M)[\text{Chain}[r + 1]] \\) \quad \text{each segment precedes the next one} \\ \text{IN} \\ [i \in \text{IndexSet}(M) \mapsto \\ \text{Max}(\{ \\ j \in 0 \dots (\text{Len}(M) - 1) : \\ \exists \text{Chain} \in [1 \dots (j + 1) \rightarrow \text{IndexSet}(M)] : \\ \text{IsDepthChain}(i, j, \text{Chain}) \quad \text{there exists a chain of length } j \text{ from } i \\ \}) \\]$$

$$d(M) \triangleq \\ \text{returns the depth of the multisegment } M \\ \text{IF } M = \langle \rangle \\ \text{THEN } 0 \\ \text{ELSE } \text{Max}(\text{Range}(\text{Depth}(M)))$$

$$\text{The Map } K$$

$DepthFiber(M, k) \triangleq$
 $Fiber(Depth(M), k)$

$d_M^{-1}(k)$

$AdmissibleEnumeration(m, k) \triangleq$
 LET
 $F \triangleq DepthFiber(m, k)$
 $IsAdmissible(P) \triangleq$
 $\forall r \in 1 \dots (Cardinality(F) - 1) :$
 $SegSubsetEq($
 $\quad Delta(m)[P[r + 1]],$
 $\quad Delta(m)[P[r]]$
 $)$
 IN
 CHOOSE $P \in Permutations(F) :$
 $IsAdmissible(P)$

depth fiber at k

P satisfies the admissibility condition

choose any admissible enumeration

$jIndex(M, k) \triangleq$
 IF $k \in Range(Depth(M))$
 THEN $Last(AdmissibleEnumeration(M, k))$
 ELSE 0

$IVee(M) \triangleq$
 LET
 $Cycles \triangleq$
 $\{AdmissibleEnumeration(M, k) :$
 $\quad k \in Range(Depth(M))\}$
 IN
 $PermutationFromCycles($
 $\quad IndexSet(M),$
 $\quad Cycles$
 $)$

one cycle for each depth fiber

permutation $i \mapsto i^{vee}$ induced by these cycles

2.1a
 $SegmentTransform(M, i) \triangleq$
 this is $_i$
 $Segment($
 $\quad b(Delta(M)[i]),$
 $\quad e(Delta(M)[IVee(M)[i]])$
 $)$

2.1b
 $DistinguishedIndices(M) \triangleq$

$$\begin{aligned}
& \{jIndex(M, k) : k \in Range(Depth(M))\} \\
RemainingIndices(M) & \triangleq \\
& \text{this is } \Gamma' = \Gamma \setminus \Gamma' \\
& IndexSet(M) \setminus DistinguishedIndices(M) \\
2.1c \\
l(M) & \triangleq \\
& [r \in 1 \dots (d(M) + 1) \mapsto \\
& \quad SegmentTransform(M, jIndex(M, r - 1)) \\
&]
\end{aligned}$$

$$\begin{aligned}
DerivedMultisegment(M) & \triangleq \\
EnumerateValues(& \\
& [i \in RemainingIndices(M) \mapsto \\
& \quad SegmentTransform(M, i)] \\
&)
\end{aligned}$$

$$m' = \text{sum of } Delta_i \text{ for } i \text{ in } \Gamma'$$

$$\begin{aligned}
2.2 \\
bTransform(M, i) & \triangleq \\
& b(SegmentTransform(M, i)) \\
eTransform(M, i) & \triangleq \\
& e(SegmentTransform(M, i)) \\
K(M) & \triangleq \\
& \langle l(M), DerivedMultisegment(M) \rangle
\end{aligned}$$

The single MW step

$$\begin{aligned}
min(M) & \triangleq \\
& Min(\{b(Delta(M)[i]) : i \in IndexSet(M)\})
\end{aligned}$$

$$\begin{aligned}
3.1a - 3.1c \\
LeadingSequence(M) & \triangleq \\
LET & \\
& CandidateSequences \triangleq \\
& \quad UNION \{ \\
& \quad \quad [1 \dots k \rightarrow IndexSet(M)] : \\
& \quad \quad k \in 1 \dots Len(M) \\
& \quad \}
\end{aligned}$$

all possible nonempty index sequences of relevant length

$$\begin{aligned}
FirstOK(i) & \triangleq \\
& \wedge \quad b(Delta(M)[i]) = min(M)
\end{aligned}$$

$Delta_i$ begins at $min \ M$

$\wedge \quad \forall q \in \text{IndexSet}(M) :$ $b(\text{Delta}(M)[q]) = \min(M)$ $\Rightarrow$ $e(\text{Delta}(M)[i]) \leq e(\text{Delta}(M)[q])$	<i>Delta_i</i> has minimal ending among these segments
$\text{NextCandidate}(i, q) \triangleq$ $\wedge \text{Precedes}(\text{Delta}(M)[i], \text{Delta}(M)[q])$ $\wedge b(\text{Delta}(M)[q]) = b(\text{Delta}(M)[i]) + 1$	$\text{Delta}_i \quad \text{Delta}_q$ <i>Delta_q</i> begins exactly one later
$\text{NextOK}(i, \text{next}) \triangleq$ $\wedge \text{NextCandidate}(i, \text{next})$ $\wedge \forall q \in \text{IndexSet}(M) :$ $\text{NextCandidate}(i, q)$ $\Rightarrow$ $e(\text{Delta}(M)[\text{next}]) \leq e(\text{Delta}(M)[q])$	next is an eligible successor next has minimal ending among eligible successors
$\text{IsValid}(S) \triangleq$ $\wedge \text{FirstOK}(S[1])$ $\wedge \forall r \in 1 \dots (\text{Len}(S) - 1) :$ $\text{NextOK}(S[r], S[r + 1])$ $\wedge \neg \exists q \in \text{IndexSet}(M) :$ $\text{NextCandidate}(\text{Last}(S), q)$	first index satisfies (3.1a) successive indices satisfy (3.1b) no successor exists after the last index: (3.1c)
IN CHOOSE $S \in \text{CandidateSequences} :$ $\text{IsValid}(S)$	choose one complete leading sequence
$\text{IStar}(M) \triangleq$ $\mathbf{I}^* = \{i_1, \dots, i_k\}$ $\text{Range}(\text{LeadingSequence}(M))$	image of the chosen leading sequence
$\text{DeltaStar}(M, i) \triangleq$ IF $i \in \text{IStar}(M)$ THEN $\text{SegTruncateLeft}(\text{Delta}(M)[i])$ ELSE $\text{Delta}(M)[i]$	if i is one of the leading indices $\text{Delta}_{i^*} = - \text{Delta}_i$ otherwise $\text{Delta}_{i^*} = \text{Delta}_i$
3.2 $\text{MCross}(M) \triangleq$ LET $\text{KeptIndices} \triangleq$ $\{i \in \text{IndexSet}(M) :$ $\text{DeltaStar}(M, i) \neq \text{EmptySeg}\}$ IN EnumerateValues($[i \in \text{KeptIndices} \mapsto$	discard empty transformed segments

) $DeltaStar(M, i]$

$m^{\wedge cross} = \text{sum of the nonempty } Delta_{-i}^*$

$DeltaCircle(M) \triangleq$
 $Segment($
 $\quad min(M),$
 $\quad min(M) + Len(LeadingSequence(M)) - 1$
 $)$

left endpoint is $min\ m$

right endpoint is $min\ m + k - 1$

$MW(M) \triangleq$
 $\langle MCross(M), DeltaCircle(M) \rangle$

$MW(m) = (m^{\wedge cross}, Delta^{\wedge circ}(m))$

Lemma 1

THEOREM $Lemma1 \triangleq$
 $\forall m \in (Multisegments \setminus \{\langle \rangle\}) :$
 $\quad min(m) < min(l(m))$
 $\Rightarrow$
 $\quad \wedge DerivedMultisegment(m) \neq \langle \rangle$
 $\quad \wedge MCross(m) \neq \langle \rangle$

PROOF OMITTED

The Statement - Corollary 3.4
 First alternative formulation

THEOREM $Corollary1 \triangleq$
 $\forall M \in (Multisegments \setminus \{\langle \rangle\}) :$
 $\quad min(M) < min(l(M))$
 $\Rightarrow$
 $\quad \wedge l(M) = l(MCross(M))$
 $\quad \wedge DeltaCircle(M)$
 $\quad \quad = DeltaCircle(DerivedMultisegment(M))$
 $\quad \wedge SameMultisegment($
 $\quad \quad \quad DerivedMultisegment(MCross(M)),$
 $\quad \quad \quad MCross(DerivedMultisegment(M))$
 $\quad)$

for every nonempty multisegment M
 assuming $min\ M < min\ l(M)$

$l(m) = l(m^{\wedge cross})$

$Delta^{\wedge circ}(m) = Delta^{\wedge circ}(m')$

$(m^{\wedge cross})' = (m')^{\wedge cross}$ as multisegments

PROOF OMITTED

The Statement - Corollary 3.4
Second alternative formulation

THEOREM *Corollary2* $\triangleq$

LET

$$KxId(P) \triangleq$$

LET

$$KP \triangleq K(P[1])$$

IN

$$\langle KP[1], KP[2], P[2] \rangle$$

$$IdxMW(P) \triangleq$$

LET

$$MWP \triangleq MW(P[2])$$

IN

$$\langle P[1], MWP[1], MWP[2] \rangle$$

$$SameResult(P, Q) \triangleq$$

$$\wedge P[1] = Q[1]$$

$$\wedge SameMultisegment(P[2], Q[2])$$

$$\wedge P[3] = Q[3]$$

IN

$$\forall M \in (Multisegments \setminus \{\langle \rangle\}) :$$

$$min(M) < min(l(M))$$

$\Rightarrow$

$$SameResult(\\begin{array}{l} KxId(MW(M)), \\ IdxMW(K(M)) \end{array})$$

)

apply K to the first component

$$(K \times Id)(P)$$

apply MW to the second component

$$(Id \times MW)(P)$$

ladder components are equal

multisegment components are equal

distinguished segments are equal

$$(K \times Id)(MW(m)) = (Id \times MW)(K(m))$$

PROOF OMITTED

The Statement - Corollary 3.4
Third alternative formulation

THEOREM *Corollary3* $\triangleq$

$$\forall M \in (Multisegments \setminus \{\langle \rangle\}) :$$

$$min(M) < min(l(M))$$

$\Rightarrow$

$$\wedge K(M)[1] = K(MW(M)[1])[1]$$

$$\wedge MW(M)[2] = MW(K(M)[2])[2]$$

$$l(m) = l(m^{\wedge cross})$$

$$Delta^{\wedge circ}(m) = Delta^{\wedge circ}(m')$$

$\wedge \text{SameMultisegment}(\text{K}(\text{MW}(\text{M})[1])[2], \text{MW}(\text{K}(\text{M})[2])[1])$
 $)$

$(m^{\wedge \text{cross}})' = (m')^{\wedge \text{cross}}$ as multisegments

PROOF OMITTED

Glossary

Segment constructs the sequence $\langle a, b \rangle$ (keeping in mind that a sequence is a function from indexes to values of the sequence). why choose a sequence and not a set? because what we mainly use and care about in the proof are the endpoints.

Seg

the set of all valid segments $[a, b]$ with $a \leq b$.

EmptySeg

the empty sequence $\langle \rangle$, which we use to represent the empty segment. *EmptySeg* is not part of *Seg*.

$b(s)$

returns the beginning/left endpoint of a segment s . if $s = \langle a, b \rangle$, then $b(s) = a$.

$e(s)$

returns the ending/right endpoint of a segment s . if $s = \langle a, b \rangle$, then $e(s) = b$.

SegShiftLeft(s)

shifts both endpoints of a segment one place to the left. $[a, b]$ becomes $[a - 1, b - 1]$.

SegTruncateLeft(s)

removes the left endpoint of a segment. if the segment is $[a, b]$ with $a < b$, it returns $[a + 1, b]$. if the segment is a singleton $[a, a]$, it returns *EmptySeg*.

Precedes(s1, s2)

checks whether $s1 \prec s2$. this means that both the beginning and the ending of $s1$ are strictly smaller than those of $s2$.

SegSubsetEq(s1, s2)

checks whether $s1$ is contained in $s2$, including equality. for $s1 = [a1, b1]$ and $s2 = [a2, b2]$, this means $a2 \leq a1$ and $b1 \leq b2$.

Multisegments

the set of all multisegments, which are sequences of segments. why choose sequences of segments and not multisets/bags of segments? because we need to easily be able to use the indexes of the segments, even with repetition.

this means that two different sequences may represent the same mathematical multisegment. we deal with this using *SameMultisegment* rather than ordinary sequence equality.

SubMultisegment(N, M)

checks whether N is a submultisegment of M . internally, this is *SubBag(N, M)*, meaning that every segment occurring in N occurs in M at least as many times.

SameMultisegment(M, N)

checks whether M and N represent the same mathematical multisegment, even if they are different sequences. internally, this is *SameBag*(M, N), so equality is determined by the segments and their multiplicities and not by their order.

IndexSet(M)

returns the set $1 \dots \text{Len}(M)$, i.e. the set of global indexes of the segments in M .

Delta(M)

returns the function $i \mapsto \text{Delta}_i$, where Delta_i is the segment at global index i . since M itself is a sequence, this is extensionally the same function as M , but we keep $\text{Delta}(M)$ in order to stay close to the notation of the paper.

IsLadder(M)

checks whether M is a ladder. first, M must be nonempty. then there must exist a permutation R of *IndexSet*(M) such that the segments are in standard form, i.e. $\text{Delta}_{R[i+1]} \leq \text{Delta}_{R[i]}$ for every successive pair. using *Permutations* here already makes sure that R contains every global index exactly once.

Depth(M)

returns the function $i \mapsto d_M(i)$, where $d_M(i)$ is the depth of the segment at index i . it does this by taking the maximum j such that there exists a depth chain of length j starting at index i .

internally, *IsDepthChain*(i, j, Chain) checks that $\text{Chain}[1] = i$ and that for every r in $[j]$, $\text{Delta}_{\text{Chain}[r]} \leq \text{Delta}_{\text{Chain}[r+1]}$. we do not define *IsDepthChain* as an outer operator because it is only auxiliary machinery needed for defining *Depth*.

$d(M)$

returns the depth of the multisegment M . if M is empty it returns 0. otherwise, it takes the maximum value in *Range*(*Depth*(M)).

DepthFiber(M, k)

returns the depth fiber $d_M^{-1}(k)$, i.e. the set of indexes i such that $\text{Depth}(M)[i] = k$. this is implemented using the general *Fiber* operator.

AdmissibleEnumeration(m, k)

chooses an admissible enumeration of the depth fiber at k . it first sets F to *DepthFiber*(m, k), and then chooses a permutation P of F such that $\text{Delta}_{P[r+1]} \leq \text{Delta}_{P[r]}$ for every successive pair. we do not impose any canonical ordering: *CHOOSE* simply selects one admissible enumeration.

jIndex(M, k)

returns j_k , i.e. the last index in the chosen admissible enumeration at depth k . if k is not a depth occurring in M , it returns 0. the last element is obtained using the general *Last* operator.

IVee(M)

returns the permutation $i \mapsto i^{\text{vee}}$. for each depth, the chosen admissible enumeration is treated as a cycle. the set of these cycles is then passed to the general *PermutationFromCycles* operator.

SegmentTransform(M, i)

returns Delta'_i . its beginning is $b(\text{Delta}_i)$, while its ending is $e(\text{Delta}_{i^{\text{vee}}})$.

DistinguishedIndices(M)

returns the set I' of distinguished indexes j_k , one for every depth occurring in M .

RemainingIndices(M)

returns $I'' = I \setminus I'$, i.e. the global indexes that are not distinguished.

$l(M)$

returns the ladder $l(m)$. for each depth $r - 1$ from 0 to $d(M)$, it takes the transformed segment $\Delta_{j-\{r-1\}}$.

$DerivedMultisegment(M)$

returns m' , the multisegment consisting of Δ_i for all i in I' . since a multisegment is represented by a sequence, we need some enumeration of these values. this is handled by the general *EnumerateValues* operator rather than being built directly into this paper-specific definition.

$K(M)$

returns $\langle l(M), DerivedMultisegment(M) \rangle$, i.e. $K(m) = (l(m), m')$.

$min(M)$

returns $min\ m$, i.e. the smallest beginning among all segments of M . this is implemented by taking Min of the set of all $b(\Delta_i)$.

$LeadingSequence(M)$

chooses a sequence $\langle i_1, \dots, i_k \rangle$ satisfying 3.1a – 3.1c.

internally, *CandidateSequences* contains all possible nonempty sequences of indexes of length at most $Len(M)$.

$FirstOK(i)$ checks 3.1a: Δ_i begins at $min(M)$, and among all segments beginning there it has minimal ending.

$NextCandidate(i, q)$ checks whether q can follow i : $\Delta_i \Delta_q$ and $b(\Delta_q) = b(\Delta_i) + 1$.

$NextOK(i, next)$ checks 3.1b: $next$ is a valid candidate, and among all valid candidates it has minimal ending.

$IsValid(S)$ checks that the first element satisfies *FirstOK*, every successive pair satisfies *NextOK*, and that there is no valid successor after the last element, which gives 3.1c.

CHOOSE then selects one complete leading sequence. these auxiliary operators are local because they are only machinery for implementing the recursive construction described in the paper.

$IStar(M)$

returns $I^* = \{i_1, \dots, i_k\}$, the set of indexes occurring in the chosen leading sequence. this is simply $Range(LeadingSequence(M))$.

$\DeltaStar(M, i)$

returns Δ_{i^*} . if i is in I^* , it applies *SegTruncateLeft* to Δ_i . otherwise, it leaves Δ_i unchanged.

$MCross(M)$

returns $m^{\wedge cross}$. first it keeps exactly those indexes for which Δ_{i^*} is not *EmptySeg*. it then uses *EnumerateValues* to turn the surviving transformed segments into a sequence. therefore empty transformed segments are discarded, while repetitions of nonempty segments are preserved.

$\DeltaCircle(M)$

returns $\Delta^{\wedge circle}$. if the chosen leading sequence has length k , then $\Delta^{\wedge circle} = [min(M), min(M) + k - 1]$.

$MW(M)$

returns $\langle MCross(M), \DeltaCircle(M) \rangle$, i.e. $MW(m) = (m^{\wedge cross}, \Delta^{\wedge circle})$.

Utils

Range(f)

returns the image/range of a function f , *i.e.* $\{f[x] : x \in \text{DOMAIN } f\}$.

Max(S)

returns the maximum element of a nonempty finite set S .

Min(S)

returns the minimum element of a nonempty finite set S .

IsInjective(f)

checks whether f is injective: distinct elements of $\text{DOMAIN } f$ must have distinct images.

Permutations(S)

returns the set of all permutations of a finite set S . a permutation is represented as a function from $1 \dots \text{Cardinality}(S)$ to S which is injective. because the domain and S have the same finite cardinality, injectivity also guarantees that every element of S occurs exactly once.

ChoosePermutation(S)

chooses one arbitrary permutation of S using `CHOOSE`. this is useful whenever we need to turn a set of indexes into a sequence but do not care about the particular order.

Fiber(f, y)

returns the fiber/preimage of y under f , *i.e.* all x in $\text{DOMAIN } f$ such that $f[x] = y$.

Last(S)

returns the last element of a nonempty sequence S , *i.e.* $S[\text{Len}(S)]$.

Multiplicity(S, x)

returns the number of indexes i for which $S[i] = x$. when S represents a bag/multiset, this is the multiplicity of x in that bag.

SubBag(A, B)

checks whether A is a subbag of B . for every value occurring in A , its multiplicity in A must be no greater than its multiplicity in B .

SameBag(A, B)

checks whether A and B represent the same bag. this means $\text{SubBag}(A, B)$ and $\text{SubBag}(B, A)$, so order is ignored but repetitions/multiplicities are preserved.

PermutationFromCycles(I, Cycles)

constructs a permutation of I from a set of cycles. for each i , it finds the cycle E containing i and the position r of i in E . if i is the last element of E , it maps i to $E[1]$. otherwise, it maps i to $E[r + 1]$.

EnumerateValues(f)

turns the values of a finite function f into a sequence. it chooses a permutation of $\text{DOMAIN } f$ and returns the values of f in that order. unlike $\text{Range}(f)$, this preserves repetitions: if two different indexes map to the same value, that value appears twice in the resulting sequence.

Notes on notation and organization

why choose operators and not function notations? for instance, why define *Multiplicity* as $\text{Multiplicity}(S, x)$ without specifying the domain, meaning something like

Multiplicity[$S \in Seq(\dots), x \in \dots$]?

TLA+ is untyped, so if we do need to specify the domain, we can do so in the definitions that use these objects. second, the bracket notation defines a function, while the operator notation defines an operator. from a more practical standpoint, unfolding the operator notation in a proof requires less machinery and less proof obligations.

the purpose of *Utils* is to separate operations that are not specific to this paper, such as *Range*, *Fiber*, permutations, bag equality, or constructing a permutation from cycles.